Image Segmentation and Morphological Processing Analysis Report

Matthew Harry Indriadi
24/532723/PA/22527

Objective

To evaluate the effectiveness of a custom, dependency-free Python implementation for segmenting a noisy image and cleaning the output using morphological operations.

Methodology

The code implements a two-step image processing pipeline using standard mathematical operations:

1. **Otsu's Thresholding:** An automatic segmentation algorithm that scans all possible pixel intensities (0-255) to find the optimal threshold. It separates the image into foreground and background by maximizing the variance between the two pixel classes.
2. **Morphological Operations:** The code applies a 3x3 structuring element (kernel) to clean the binary image.
    - **Opening (Erosion followed by Dilation):** Used to remove isolated bright pixels (salt noise) from the dark background.
    - **Closing (Dilation followed by Erosion):** Used to fill in small dark holes (pepper noise) within the bright foreground object.

Results Analysis

The visual results of the pipeline demonstrated that the algorithm calculated an optimal Otsu threshold value of 99.

- **Original Noisy Image:** The raw input displays a central bright square that is heavily obscured by simulated background interference and severe salt-and-pepper noise.
- **Otsu Segmentation (Without Morphology):** While the thresholding step successfully isolates the primary square shape, the resulting binary mask is highly

degraded. The background is speckled with false-positive white pixels, and the central square contains numerous false-negative black holes.

- **Otsu Segmentation (+ Opening & Closing):** The application of morphological processing significantly improves the final output. The opening operation successfully strips away the background noise, while the closing operation fills almost all internal gaps within the central object. The final result is a solid, distinct white square against a clean black background, with only minor edge artifacts remaining.

Conclusion

The custom implementation demonstrates that while Otsu's thresholding is mathematically effective at separating distinct object classes, it is highly sensitive to pixel-level noise. However, pairing it with basic morphological operations (opening and closing) creates a highly robust and accurate segmentation pipeline without the need for heavy external computer vision libraries.